BUSINESS ACADEMY AARHUS

Exercises. Machine Learning. Week 8.

Sila, Oct 21st, 2026.

Exercise 1.

See the StudentChurn.pdf document on Canvas.

Exercise 2.

Getting started with convolutional layers.

Use a single convolutional layer to detect a line in some input data.

See the ML_Week_11_Exercise2_Convolution.py for a demo of the calculations.

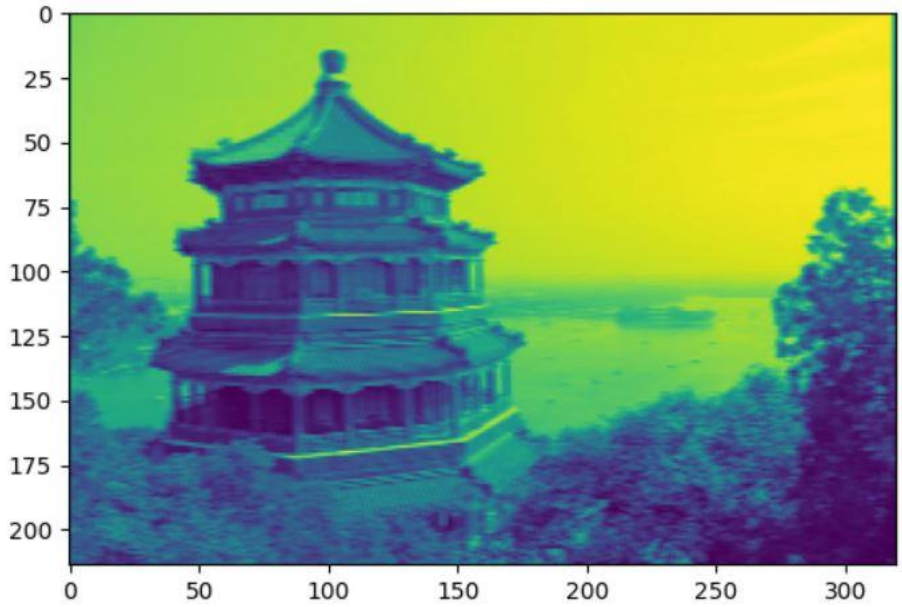
Exercise 3.

In this exercise you should run the example from the book p. 491 (v2. p. 453).

See explanation there (ML_Week_11_Exercise3_Convolution.py).

Change the strides parameter to 10, and see what happens.

Look at output for both pictures.



Exercise 4.

In groups of 2-3**.**

In this exercise we will try to classify the CIFAR-10 dataset, a collection of
60.000 32x32-pixel color images, that is included in the deeplearning framework Keras.
Each image is classified into exactly 10 classes, and the task now is to learn these classifications.

Look at the code in the file ML_Week_11_Exercise4_DeepLearning.ipynb

Look up the meaning of the Keras concepts:

- Flatten
- Dense layer.

And explain them to each other.

Then run the code.
You are likely to end up with a result of around 50 %.

- What does this mean? Is it better than random guessing?

Extra:
Test another loss function, and see if that helps. Read about it here:
https://keras.io/api/losses/

Exercise 5.

In groups of 2-3**.**

We will then try to add convolutional layers to the model we used in the previous exercise.

See the code in the file ML_Week_11_Exercise5_Conv_DeepLearning.ipynb

Explain the model in details to each other, starting from how the input comes into the model, and then moving from layer to layer.

Make a sketch of what a layer like this:

```
conv_layer_1 = Conv2D(
   filters = 10
   , kernel_size = (4,4)
   , strides = 2
   , padding = 'same'
   )(input_layer)
```

Would look like, and what it would do.

Run the code.

Apparently adding (naively) just these 2 convolutional layers didn't help.

Try with just one convolution. Did that help?

Try out different learning_rates and number of epochs in order to get a feel for how close or how far we are from some that could be a big improvement compared to the results in the previous exercise.

Exercise 6.

In groups of 2-3**.**

Finally, we will not only add convolutional layers (as we did in exercise 5), but also use batch-normalization and explicitly use the Leaky ReLu activation function.

See the code in the file ML_Week_11_Exercise6_Conv_DeepLearning.ipynb

Running the notebook will probably take a little (a lot) longer but show a significant improvement in the result. A result of around 70% or better is expected.

// expect 100 – 150 sec. per epoch, depending on laptop //

Try to remove some of the convolutions. E.g.

```
x = Conv2D(filters = 64, kernel_size = 3, strides = 1, padding = 'same')(x)
```

```
x = BatchNormalization()(x)
x = LeakyReLU()(x)

x = Conv2D(filters = 64, kernel_size = 3, strides = 2, padding = 'same')(x)
x = BatchNormalization()(x)
x = LeakyReLU()(x)
```

Adjust the strides parameter in a layer (remember that increasing it will give less output to the next layer). And try with and without the dropout layer (which supposedly work as a regularization layer).

We will talk about your findings next time/when everyone in class have tried (misc settings here).

Exercise 7.

In groups of 2-3**.**

Work with chatGPT to come up with your own example of using convolutions.

What does chatGPT suggest that you should do? Does it work?